

Dilema celor doi agenti

- **Limită de timp:** 12 minute.
- **Spațiu de stocare:** 5 GB
- **Mediu:** un GPU (≈ 16 GB VRAM), fără internet
- **Dimensiunea soluției:** `solution.ipynb` ≤ 1 MB
- **Scor baseline:** 0

La centrul național de AI din Astana, două modele de calculator — Model R (un ResNet-18) și Model V (un ViT-Tiny) — analizează fotografii. În acest moment, ambele modele funcționează perfect, obținând o acuratețe de 100% și fiind de acord pentru fiecare imagine în parte. Pentru a testa cât de diferite sunt în realitate „creierile” lor inteligente, cercetătorul-șef vă propune o provocare: efectuați modificări minuscule, aproape invizibile, ale pixelilor fiecărei fotografii, astfel încât Model R și Model V să fie în dezacord complet.



1. Sarcină

Două clasificatoare de imagini preantrenate analizează aceeași imagine. Pe imaginile furnizate în această sarcină, ambele clasificatoare obțin o acuratețe de 100%.

- **Model R:** `torchvision.models.resnet18` (un CNN, ResNet18).
- **Model V:** `timm's vit_tiny_patch16_224` (un Transformer, ViT-Tiny).

Sarcina dumneavoastră este să creați câte o mică modificare („perturbație”) pentru fiecare imagine, astfel încât cele două modele să fie în dezacord. Pentru fiecare imagine, trebuie să creați **două perturbații diferite**:

- **Tipul A:** după adăugarea acesteia, Model R clasifică în continuare imaginea corect, dar Model V o clasifică incorect.
- **Tipul B:** după adăugarea acesteia, Model V clasifică în continuare imaginea corect, dar Model R o clasifică incorect.

Fiecare perturbație trebuie să fie suficient de *mică* încât să fie greu de observat. Perturbațiile mai mici obțin scoruri mai mari (consultați Secțiunea 5). Perturbația este aplicată direct imaginii originale, la nivel de pixel.

2. Date publice

Împreună cu sarcina este furnizat un set de imagini, organizat în două subseturi — `train` (100 imagini) și `test_public` (100 imagini) — fiecare conținând imagini cu rezoluții diferite. Toate imaginile provin din cele 1000 de clase ale ImageNet-1K, iar atât Model R, cât și Model V obțin o acuratețe de 100% pe ambele subseturi.

Sunt furnizate următoarele fișiere:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

În timpul evaluării, folderul dumneavoastră `dataset/test_public/` este înlocuit în mod transparent cu două seturi ascunse de imagini (`test_leaderboard_a` și `test_leaderboard_b`) pentru calcularea scorului oficial. Fiecare dintre acestea conține **100 imagini** în format PNG și un fișier cu etichete.

Notă: Pentru această sarcină, etichetele din dataseturile de test sunt accesibile.

3. Formatul rezultatului

Pentru fiecare imagine, trebuie să produceți două fișiere:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), corespunde numelui imaginii din dataseturi.

- Fiecare fișier este un singur tensor salvat cu `torch.save`. Forma sa trebuie să fie $3 \times H \times W$, unde H și W corespund rezoluției **originale** a imaginii respective (nu 224×224).
- Codul trebuie să producă un singur fișier ZIP, `submission.zip`. Plasați toate fișierele `.pt` la nivelul superior al arhivei ZIP, fără un folder exterior sau subdirectoare.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Notebookul vă va avertiza dacă există vreo probleme privind formatul rezultatului.

4. Constrângeri

- **Modele:** Trebuie să utilizați `torchvision.models.resnet18(pretrained=True)` și `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Nu sunt permise alte modele preantrenate.
- **Pipeline de transformare (impus la evaluare):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` pentru detalii.
- **Rezoluția perturbației:** Trebuie să corespundă rezoluției **originale** a imaginii brute (nu 224×224). Tensorul este adăugat imaginii brute *înaintea* pipeline-ului de transformare.
- **Formatul rezultatului:** Numai fișiere `.pt` — fără PNG/JPG. Tensorii sunt adăugați imaginii brute, iar valorile pixelilor sunt limitate la $[0, 1]$ înainte de preprocesare.
- **Denumirea fișierelor:** Listare fără ierarhie, în formatul strict `{index}_a.pt` / `{index}_b.pt`. Fără subdirectoare în interiorul arhivei zip.
- **Biblioteci:** `torch`, `torchvision`, `timm`.

5. Punctaj

Scorul final este calculat după cum urmează. Fie M numărul de imagini din subset, $Score_A$ numărul perturbațiilor de Tip A reușite și $Score_B$ numărul perturbațiilor de Tip B reușite:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF este o funcție concepută pentru a penaliza perturbațiile cu o normă mare și pentru a fi foarte sensibilă în apropierea plafonului performanței. Ea este limitată la intervalul de la 0.5 la 1. Implementarea completă poate fi consultată în Secțiunea 8 din `solution.ipynb`.

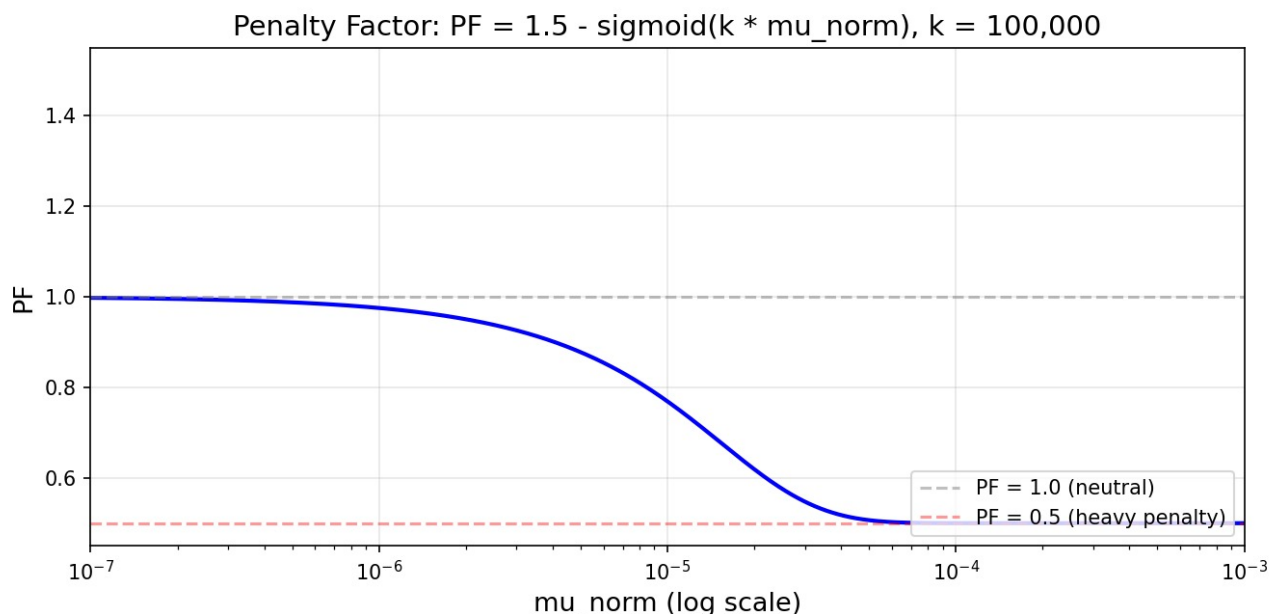


Figura: Curba funcției de penalizare.

6. Verificarea trimiterii

În notebook există verificări care vă avertizează dacă există probleme de formatare, în Secțiunea 7 din notebookul `solution.ipynb`.

7. Testare locală

`solution.ipynb` conține un exemplu complet și funcțional. Acesta încarcă datele publice, ambele modele și evaluatorul oficial și scrie un fișier ZIP de trimitere. Citiți-l înainte de a începe.

8. Cum să trimiteți

- Salvați modificările în `solution.ipynb`.
- Deschideți fila Git din bara laterală stângă a JupyterLab.
- Efectuați **Stage** pentru `solution.ipynb` (pictograma + de lângă acesta).
- Introduceți un mesaj de commit și faceți clic pe **Commit**.
- Faceți clic pe pictograma nor cu săgeată în sus pentru a efectua push.
- Reveniți la această pagină a concursului și faceți clic pe **Submit**.

Trimiteți exact un fișier, denumit `solution.ipynb`.